

Домашнее задание

Assembly + Binary Bomb

Пишем простые программы на ассемблере x86-64 и аккуратно разбираем учебную binary bomb.

Дисклеймер про ИИ

Мы любим ИИ и активно используем его в обучении. Но в этой домашке цель - потренировать именно ваше понимание ассемблера, ABI, регистров и отладчика. Поэтому не используйте ChatGPT, Copilot и другие ИИ-инструменты для решения задач и прохождения бомбы.

Общие правила

- Работаем с AT&T-синтаксисом, то есть без -Mintel.
- Ассемблерные файлы называем hello.s, sum.s, fib.s.
- Собираем через clang, без nasm.
- Можно использовать libc: puts, printf, scanf и обычную точку входа main.
- Продвинутый вариант - писать без libc через Linux syscall и точку входа _start.

```
# Типовая сборка .s файла через clang
clang -no-pie hello.s -o hello
./hello

# Если вы пишете программу со своей точкой входа _start и без libc
clang -nostdlib -no-pie hello_syscall.s -o hello_syscall
```

Почему -no-pie?

Так адреса в objdump/gdb/lldb будут стабильнее и понятнее для первых упражнений. Это учебное упрощение, а не требование безопасности.

Задача 1. Hello, world!

Напишите программу на ассемблере x86-64 под Linux, которая выводит строку:

```
Hello, world!
```

- Файл: `hello.s`.
- Можно использовать `libc`, например вызов `puts` или `printf`.
- Можно сделать вариант без `libc` через `write` и `exit` syscall.
- Программа должна собираться через `clang` и запускаться на сервере.

```
clang -no-pie hello.s -o hello
./hello
```

Что важно понять

Как объявить секции `.text` и `.rodata`, как передать аргумент в функцию по ABI и чем `main` отличается от `_start`.

Задача 2. Сложение

Напишите программу на ассемблере, которая складывает два целых числа.

- Можно захардкодить числа прямо в программе.
- Положите значения в регистры и используйте инструкцию `add`.
- Результат можно вывести через `libc` или вернуть как `exit code`.
- В комментариях к коду напишите, какие регистры вы использовали и зачем.

```
clang -no-pie sum.s -o sum
./sum
```

```
echo $? # если вы возвращаете результат как exit code
```

Без спойлеров

Не копируйте готовый код. Цель задачи - руками пройти путь: значения -> регистры -> инструкция сложения -> результат.

Задача 3. Fibonacci

Напишите программу на ассемблере, которая вычисляет число Фибоначчи для выбранного в задании индекса.

- Решение должно считать значение, а не просто возвращать заранее записанную константу.
- Используйте цикл и регистры для хранения промежуточных значений.
- Рекурсия не обязательна: итеративный вариант проще и лучше подходит для первого asm-кода.
- Результат можно вывести через `libc` или вернуть как `exit code`.

```
clang -no-pie fib.s -o fib
./fib

echo $? # если результат возвращается как exit code
```

Что важно понять

Цикл, условный переход, обновление нескольких регистров и аккуратное завершение программы. В отчёте кратко опишите, где у вас хранится текущее и следующее значение.

Задача 4. Binary Bomb

Нужно подключиться к удалённому серверу и пройти учебную `binary bomb`. Бомба у всех одинаковая, но каждый работает под своим пользователем.

```
ssh YOUR_LOGIN@94.198.219.228
```

Пароль нужно попросить у преподавателя лично. После входа бинарь лежит в домашней директории.

```
ls -la
file bomb_hard
./bomb_hard
```

Рекомендуемый бинарь

Для домашней работы используйте `~/bomb_hard`. Он собран без `debug info`, поэтому придётся читать дизассемблер и пользоваться отладчиком, а не смотреть исходные строки.

Рекомендуемый инструмент: GDB

Для Linux x86-64 в этом задании рекомендуется использовать gdb. Он быстрее и предсказуемее на сервере, чем lldb.

```
gdb ./bomb_hard
```

Команда	Что делает
run или r	запустить программу
start	запустить и остановиться в main
break main	поставить breakpoint на main
break phase_1	поставить breakpoint на функцию
break *phase_1+26	поставить breakpoint по смещению от функции
info breakpoints	посмотреть breakpoints
delete N	удалить breakpoint с номером N
disassemble phase_1	показать ассемблер функции в AT&T-синтаксисе
info registers	показать регистры
x/s ADDRESS	прочитать строку по адресу
x/16gx ADDRESS	посмотреть 16 восьмибайтных значений памяти
si	шагнуть внутрь следующей инструкции
ni	шагнуть следующую инструкцию без захода в вызов
continue или c	продолжить выполнение
quit	выйти из gdb

Полезные команды вне GDB

```
file bomb_hard
strings bomb_hard
objdump -d bomb_hard | less
readelf -h bomb_hard
nvim notes.txt
```

Важно

Мы используем AT&T-синтаксис, поэтому не добавляйте -Intel в objdump.

Правила работы на сервере

- Не пытайтесь получить root-доступ.
- Не заходите в чужие домашние директории и не трогайте чужие файлы.
- Не мешайте другим студентам: не запускайте форк-бомбы и тяжёлые бесконечные процессы.
- Не публикуйте ответы от бомбы в общем чате. Бомба одинаковая для всех.
- В отчёте важно объяснить ход анализа, а не только прислать финальные ответы.

Что нужно сдать

- `hello.s`
- `sum.s`
- `fib.s`
- ответы для прохождения `bomb_hard`
- короткое описание, как вы нашли ответы: какие команды использовали, где ставили breakpoints, что увидели в дизассемблере.

Формат отчёта по бомбе

Не нужно присылать большой роман. Достаточно кратко: какие функции смотрели, какие сравнения или переходы нашли, какие команды GDB помогли. Не вставляйте готовые куски исходника, которого у вас нет.

Главная цель домашки - научиться читать машинный код как текст: видеть регистры, вызовы функций, условные переходы и постепенно восстанавливать логику программы.